

第9次课程-自制大模型推理框架-显存的管理

自制大模型推理框架

带你从零写一个支持LLama2/3推理支持Cuda加速和int8量化的大模型框架

课程亮点

一、项目整体架构和设计

学习架构思维，我们将在这里学习算子、张量等关键数据结构的设计。

二、支持LLama2模型结构

对LLama模型的权重读取和构建，用KV Cache等机制支持加速推理。

三、模型的量化

为了减少显存的占用，我们会一起开发int8模型量化模块，包括量化模型导出和量化算子支持。

四、Cuda基础和算子实现

带你学Cuda并实战大模型算子的实现，每个算子都将讲解原理，从零手写。

五、用推理框架做点有趣的事情

文本生成，生成故事，多轮对话。



现价优惠 扫码马上报名

视频

2024-07-21 13-11-21.mkv

回顾

我们在前文中讲到，在不同的设备上为了屏蔽底层内存申请、释放和拷贝方法接口的不同，我们设计了 `Allocator` 父类，我们在不同设备上都要实现该父类的所有接口。

```
1 class DeviceAllocator {
2     public:
```

```

3     explicit DeviceAllocator(DeviceType device_type) :
device_type_(device_type) {}
4
5     virtual DeviceType device_type() const { return device_type_; }
6
7     virtual void release(void* ptr) const = 0; // 要实现的内存分配接口
8
9     virtual void* allocate(size_t byte_size) const = 0; // 要实现的内存释放接口
10
11    virtual void memcpy(const void* src_ptr, void* dest_ptr, size_t
byte_size,
12                      MallocKind malloc_kind =
MallocKind::kMallocCPU2CPU, void* stream = nullptr, bool
need_sync = false) const;
13
14    virtual void memset_zero(void* ptr, size_t byte_size, void*
stream, bool need_sync = false);
15
16    private:
17        DeviceType device_type_ = DeviceType::kDeviceUnknown;
18 };

```

其中最重要的是 `allocate` 和 `release` 接口，用于在不同设备上申请和释放内存，换句话说就是要对不同的 `Allocator` 子类都实现这两个方法。我们先来看一下CPU设备上两个接口的实现：

```

1 void* CPUDeviceAllocator::allocate(size_t byte_size) const {
2     if (!byte_size) {
3         return nullptr;
4     }
5     data = malloc(byte_size);
6     return data;
7 }
8
9
10 void CPUDeviceAllocator::release(void* ptr) const {
11     if (ptr) {
12         free(ptr);
13     }
14 }

```

可以看到就只是简单的调用了两个系统接口，没有任何多余的动作。使用方法如下所示：

```

1 void * ptr = allocator->allocate(byte_size); // 申请byte_size字节大
小的内存
2 allocator->release(ptr); // 释放申请的内存空间

```

GPU Allocator的实现

我们先来看一个最简单的实现：

```

1 void* CUDADeviceAllocator::allocate(size_t byte_size) const {
2     if (!byte_size) {
3         return nullptr;

```

```
4     }
5     void* ptr = nullptr;
6     cudaError_t err = cudaMalloc(&ptr, byte_size);
7     CHECK_EQ(err, cudaSuccess);
8     return ptr;
9 }
10
11 void CUDADeviceAllocator::release(void* ptr) const {
12     if (ptr) {
13         cudaFree(ptr);
14     }
15 }
```

CUDADeviceAllocator 是 Allocator 父类的另一个子类，表示显卡设备上的内存申请和释放器。最简单实现就是直接调用两个运行时方法 cudaMalloc 和 cudaFree 来申请释放显存。

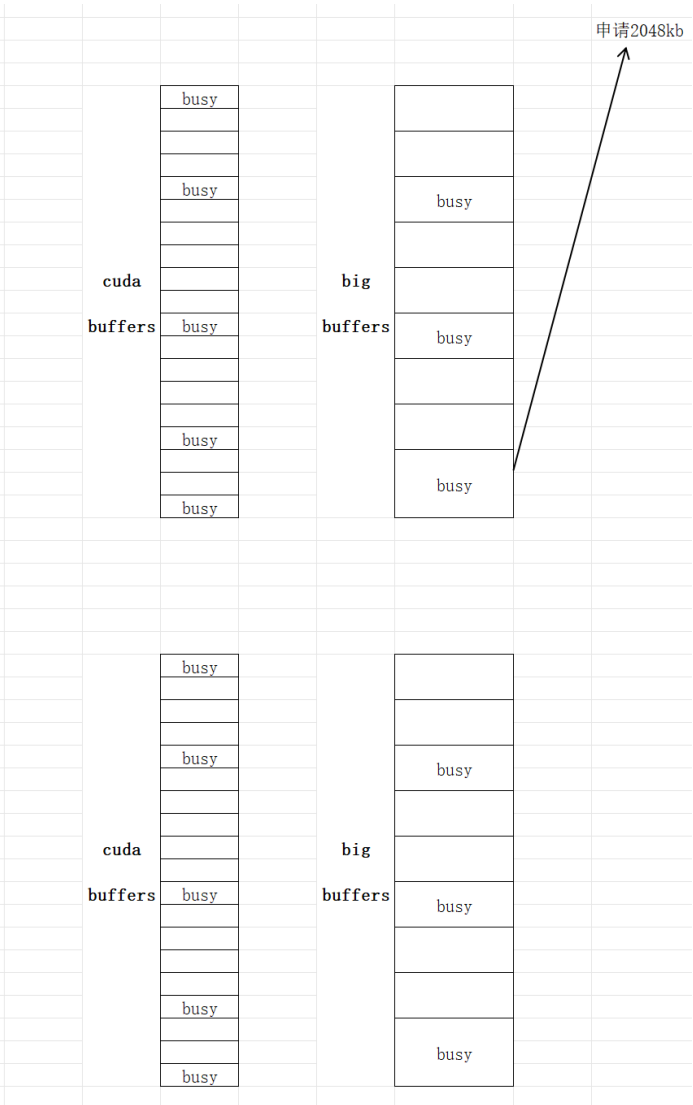
更复杂的实现

显存申请方式

cudaMalloc等系统接口有一定耗时，所以我们需要再一定程度上避免重复的申请和释放。对于小块显存，不先cudaFree还给系统，而是先保存起来，以后用到的时候直接返回该块显存既可。

- 我们将内存申请的情况分为两种，一种是申请大小大于1MB的情况，我们在使用 cudaMalloc 分配之后还需要将它记录到全局变量 big_buffers 中，另外如果 big_buffers 数组中有符合大小需求的内存块且无使用者就直接返回该内存块。

当所需的大块内存2048kb被分配前，我们会先big_buffers里找空闲且大小大于2048kb的内存块



```
1 void* CUDADeviceAllocator::allocate(size_t byte_size) const {
2     int id = -1;
```

需要再

 王丹 5:00 AM Feb 3
需要在

2048kb

 毛昊 Nov 3, 2024
是否应为 1024kb?

 王丹 5:04 AM Feb 3
没错。只要是大于等于 1MB 的显存就属于 big_buffers。这里以 2MB 为例进行说明的。

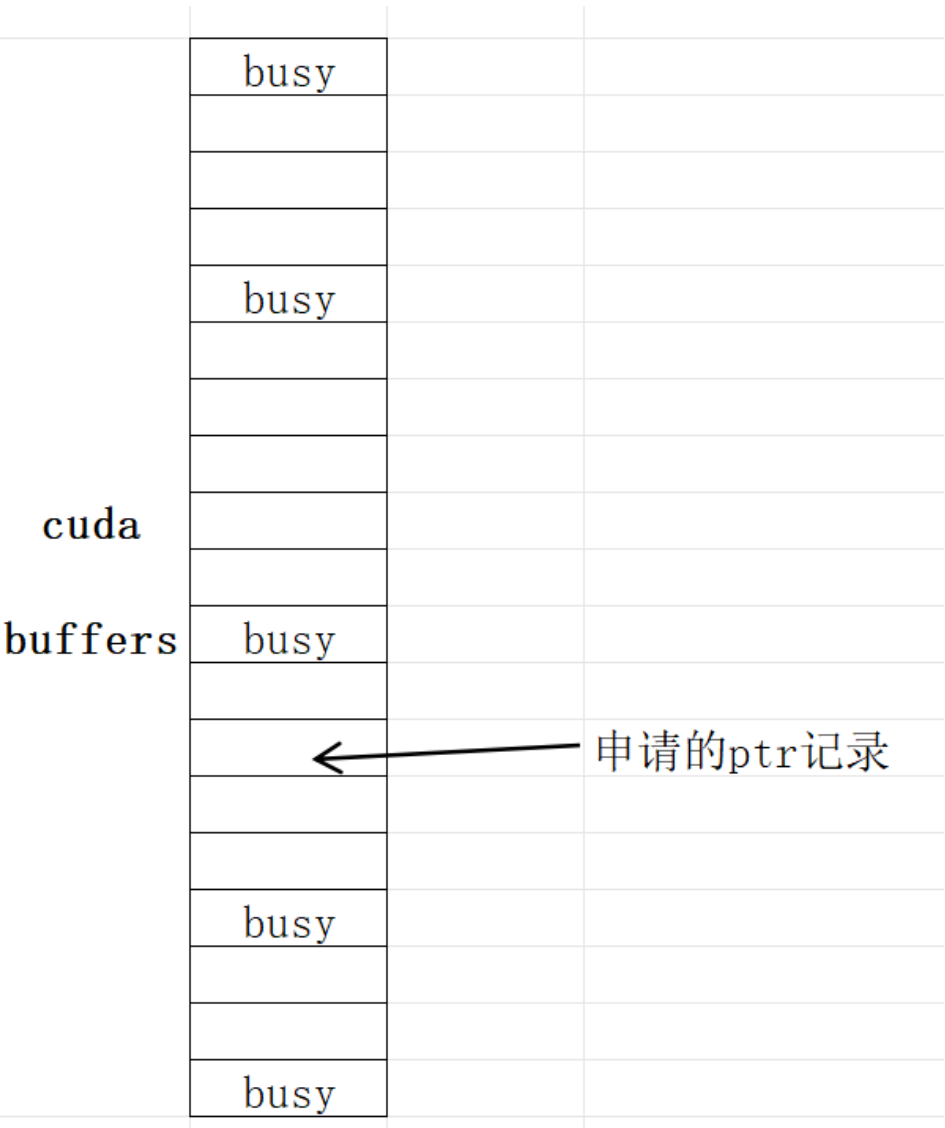
```

3     cudaError_t state = cudaGetDevice(&id);
4     CHECK(state == cudaSuccess);
5     if (byte_size > 1024 * 1024) { // 申请的字节数大于1mb
6         auto& big_buffers = big_buffers_map_[id]; //
7         int sel_id = -1;
8         for (int i = 0; i < big_buffers.size(); i++) {
9             // 遍历big_buffers里的每一个项目，满足要求的（内存块的大小大
            于所需要，busy是false）
10            if (big_buffers[i].byte_size >= byte_size &&
            !big_buffers[i].busy &&
11                big_buffers[i].byte_size - byte_size < 1 * 1024 *
            1024) {
12                // 在big_buffer数组找到的显存块大小满足要求
13                if (sel_id == -1 || big_buffers[sel_id].byte_size
            > big_buffers[i].byte_size) {
14                    sel_id = i;
15                }
16            }
17        }
18        // 直接返回找到符合的要求的项
19        if (sel_id != -1) {
20            big_buffers[sel_id].busy = true;
21            return big_buffers[sel_id].data;
22        }
23
24        // 如果没找到空闲的，再去调用cudaMalloc
25        void* ptr = nullptr;
26        state = cudaMalloc(&ptr, byte_size);
27        if (cudaSuccess != state) {
28            char buf[256];
29            snprintf(buf, 256,
30                    "Error: CUDA error when allocating %lu MB
            memory! maybe there's no enough memory "
31                    "left on device.",
32                    byte_size >> 20);
33            LOG(ERROR) << buf;
34            return nullptr;
35        }
36        big_buffers.emplace_back(ptr, byte_size, true);
37        return ptr;
38    }

```

从以上代码看出，如果申请的显存字节数大于1MB，那么我们就先从 `big_buffers` 数组中找到无人使用（`busy` 等于 `false`）且大于所需显存字节的内存块，就直接返回。如果没找到空闲的才会使用 `cudaMalloc` 系统接口，去申请对应字节的显存块并将它放到 `big_buffers` 数组中。

- 如果是小块的显存，同理我们有一个 `cuda_buffers_` 来记录已经分配出去的显存块，同样地如果 `cuda_buffers_` 中有空闲空间满足内存申请者的需要（空闲显存块 > 申请需要）就直接返回，否则才会使用 `cudaMalloc` 系统接口去分配显存块，并将它放到 `cuda_buffers` 数组中做记录，就想下图所示的那样。



想

毛昊

毛昊 Nov 3, 2024

typo

```
1  auto& cuda_buffers = cuda_buffers_map_[id];
2  for (int i = 0; i < cuda_buffers.size(); i++) {
3      if (cuda_buffers[i].byte_size >= byte_size &&
4          !cuda_buffers[i].busy) {
5          cuda_buffers[i].busy = true;
6          no_busy_cnt_[id] -= cuda_buffers[i].byte_size; // 在申请的
7          // 回收，有一块表中的显存块被申请了，空闲的数量也需要减少
8          return cuda_buffers[i].data;
9      }
10 }
11 void* ptr = nullptr;
12 state = cudaMalloc(&ptr, byte_size);
13 if (cudaSuccess != state) {
14     char buf[256];
15     snprintf(buf, 256,
16              "Error: CUDA error when allocating %lu MB memory!
17              maybe there's no enough memory "
18              "left on device.",
19              byte_size >> 20);
20     LOG(ERROR) << buf;
21     return nullptr;
22 }
23 cuda_buffers.emplace_back(ptr, byte_size, true); // 将申请到的“小”块
24 // 显存放到cuda_buffers表中
```

可以从代码的第3-6行看出，如果 `buffer_maps_` 中有符合需要的显存块就直接返回并将这一块的忙闲位 `busy` 置位true，表示该块显存已经被使用。如果没有的话，同样需要调用 `cudaMalloc` 系统接口去申请显存，并放到 `cuda_buffers` 中。

内存释放

我们从显存申请的流程可以知道，当我们需要申请一小块显存的时，还会被记录在一个类似”使用表“的结构 `buffer_maps_` 中，并将其中的对应项设为忙位（也就是 `busy` 等于true）。

在随后的显存释放中我们会将释放内存的指针和表项目中进行对比，如果是 `buffer_maps_` 的一项，**直接将 `busy` 位设置为false就行**，而不需要调用系统接口 `cudaFree` 释放显存。

cuda buffers	busy		
	busy		
	busy		
	Ptr, busy	等于	要释放显存的Ptr
	busy		
	busy		
cuda buffers	busy		
	busy		
	busy		
	busy		
	busy		

```

1 void CUDADeviceAllocator::release(void* ptr) const {
2     // ...
3     // ...
4     // 以上为清空buffer_maps_的情况
5     for (auto& it : cuda_buffers_map_) {
6         auto& cuda_buffers = it.second;
7         for (int i = 0; i < cuda_buffers.size(); i++) {
8             if (cuda_buffers[i].data == ptr) {
9                 no_busy_cnt_[it.first] +=
10                 cuda_buffers[i].byte_size; // 空闲的内存大小加释放的byte_size大小
11                 cuda_buffers[i].busy = false;
12                 return;
13             }
14         }
15     }
16 }

```

如果现在系统申请频率大于释放，就会使得 `buffer_maps_` 中空闲显存块越来越多，整个系统可以被 `cudaMalloc` 的显存就慢慢不足，所以在满足一定阈值（大于 $1024 * 1024 * 1024$ ）的情况下我们还是需要清空 `buffer_maps_` 中记录的显存项的。

```

1 void CUDADeviceAllocator::release(void* ptr) const {
2     for (auto& it : cuda_buffers_map_) {
3         if (no_busy_cnt_[it.first] > 1024 * 1024 * 1024) {
4             auto& cuda_buffers = it.second; // 找到要清理的
5             cuda_buffers
6             std::vector<CudaMemoryBuffer> temp;
7             for (int i = 0; i < cuda_buffers.size(); i++) {
8                 if (!cuda_buffers[i].busy) { // 如果这个表项为空闲
9                     state = cudaSetDevice(it.first);
10                    state = cudaFree(cuda_buffers[i].data);
11                    CHECK(state == cudaSuccess)
12                    << "Error: CUDA error when release memory
13                    on device " << it.first;
14                } else {
15                    temp.push_back(cuda_buffers[i]);
16                }
17            }
18            cuda_buffers.clear();
19            it.second = temp;
20            no_busy_cnt_[it.first] = 0; // 在清理完毕之后置为0
21        }
22    }
23 }

```

从第3行可以看出，判定的阈值条件是如果 `buffer_maps_` 中记录空闲(`busy` 等于 `false`)的显存块占用的显存字节带下大于1GB，那么我们就逐个的使用 `cudaFree` 清空 `buffer_maps_` 中的显存。

```

1 no_busy_cnt_[it.first] > 1024 * 1024 * 1024

```

我们再来看看这里的 `no_busy_cnt_` 是什么：当我们还回去一块显存的时候，除了将对应显存项位置为`false`表示当前项指向的显存块是空闲的，还会将 `no_busy_cnt` 的值加上该块显存的大小，当 `no_busy_cnt` 的大小（也就是当前空闲内存）大于阈值的时候就会像上面所说的那样进行清空。另外，对于大块显存，也就是 `big_buffers` 中记录的显存块，直接归还显存而不做特殊处理。


```

1 void CUDADeviceAllocator::release(void* ptr) const {
2     // 处理buffer_maps_中的显存块
3     auto& big_buffers = big_buffers_map_[it.first];
4     for (int i = 0; i < big_buffers.size(); i++) {
5         if (big_buffers[i].data == ptr) {
6             big_buffers[i].busy = false;
7             return;
8         }
9     }
10
11     state = cudaFree(ptr);
12 }

```

我们来简单梳理一下以上的流程：

1. 当申请显存块大小的时候，按照大小分为大块显存和小块显存，如果系统中有空闲的显存块且大小符合规则，且没有被占用，就直接返回对应显存块。如果没有的话就重新申请一块，并按照大小放在 `big_buffers_` 或者 `cuda_buffers_` 中两个结构中以供记录。
2. 当释放显存的时候，我们根据显存块的大小来将 `big_buffers` 或 `cuda_buffers` 中的某项置为空闲。如果空闲的内存块太多，我们则会集中释放一批显存块来降低系统对显存的占用。
3. 等到空闲显存块(cuda buffers数组中)的字节总数大于1gb，才会开始清理。

张量数据的互相转换

为了方便 `tensor` 中数据的互转，我们打算开发 `to_cuda` 方法和 `to_cpu` 方法。我们先来回忆一下 `tensor` 类的定义：

```

1 class Tensor {
2     public:
3         // ...
4         // ...
5     private:
6         size_t size_ = 0;
7         std::vector<int32_t> dims_;
8         std::shared_ptr<base::Buffer> buffer_; // buffer在初始化的时候，调用构造函数。最后tensor相关的数据指针是buffer->ptr
9         base::DataType data_type_ = base::DataType::kDataTypeUnknown;
10 };

```

`buffer_` 中记录的 `ptr` 指针既可以指向一块显存，也可以指向一块内存。 `buffer` 在被初始化的时候，会根据其设备的类型调用对应设备上的内存分配器。

```

1 Buffer::Buffer(size_t byte_size, std::shared_ptr<DeviceAllocator>
  allocator, void* ptr,
2             bool use_external) // 构造函数
3     : byte_size_(byte_size),
4       allocator_(allocator),
5       ptr_(ptr),
6       use_external_(use_external) {
7     if (!ptr_ && allocator_) {
8         device_type_ = allocator_->device_type();
9         use_external_ = false;
10        ptr_ = allocator_->allocate(byte_size); // 根据tensor的类型，
  buffer去调用对应设备的内存分配器去分配内、显存

```

| if (big_buffers[i].data == ptr) { big_buff...



黄梦珂 11:04 PM Feb 25

是不是少了
cudaFree(big_buffers[i].data)



飞书用户4977BL 3:50 PM Mar 18

这块不是归还这块缓存到缓存池吗，直接 free 这个空间就没了吧。直接 free 掉的话，再重新使用时还要 malloc



黄梦珂 11:29 AM Mar 19

@飞书用户 4977BL 您说得对，我当时没有理解这个内存池。



刘涵祚 5:14 PM Jul 29

我不理解这个地方的实现. 1.如果希望的效果是直接归还空间的话 big_buffer 就失去意义了 2.如果不直接归还而是记为 false 的话,没有一个类似 cuda_buffer 的清理机制来定期 cudafree


```
11     }
12 }
```

如果当前 `device_type` 表示 CPU 类型，那么 `allocator->allocate` 方法会调用 `malloc` 方法去分配内存，并以 `ptr` 指向这块被新分配的内存中。如果 `device_type` 是 GPU 类型，那么 `allocator->allocate` 就会调用前文的流程去申请一块显存，同时 `ptr` 会记录新分配的显存位置。

那么如果我们要将一个 CPU 类型的 Tensor 变为一个 GPU 类型的 Tensor，我们需要完成什么工作呢？我们对照着流程来看 Tensor 类的 `to_cuda` 方法。

1. 申请一块和原来 CPU 上占用字节大小相同的显存的 Cuda Buffer；

```
1 void Tensor::to_cuda(cudaStream_t stream) {
2     const base::DeviceType device_type = this->device_type();
3     size_t byte_size = this->byte_size();
4     auto cu_alloc =
5     base::CUDADeviceAllocatorFactory::get_instance();
6     auto cu_buffer = std::make_shared<base::Buffer>(byte_size,
7     cu_alloc); // 去申请byte_size字节大小的显存
8     cu_alloc->memcpy(buffer->ptr(), cu_buffer->ptr(), byte_size,
9     base::MemcpyKind::kMemcpyCPU2CUDA,
10    stream);
```

我们先得到 CUDA 设备上的内存分配器，也就是我们在前文中实现的 `CUDADeviceAllocator` 的一个实例，随后再用它分配和原来 `tensor` 占用字节数大小的显存块 `cu_buffer`。

2. 将原先CPU张量上的数据（保存于 `buffer_`）拷贝到新创建的显存块 `cu_buffer` 上，所以现在我们调用 `cu_alloc` 的 `memcpy` 方法

```
1 cu_alloc->memcpy(buffer->ptr(), cu_buffer->ptr(), byte_size,
2     base::MemcpyKind::kMemcpyCPU2CUDA,
3     stream); // 把原先cpu上的数据同步到了cuda上
```

`buffer->ptr()` 表示原来CPU上的内存块指针，`cu_buffer->ptr()` 表示新创建的显存块指针，显存的拷贝方向是从 CPU 到 CUDA。

```
1 void DeviceAllocator::memcpy(const void* src_ptr, void* dest_ptr,
2     size_t byte_size,
3     MemcpyKind memcpy_kind, void*
4     stream, bool need_sync) const {
5     CHECK_NE(src_ptr, nullptr);
6     CHECK_NE(dest_ptr, nullptr);
7     if (!byte_size) {
8         return;
9     }
10    cudaStream_t stream_ = nullptr;
11    if (stream) {
12        stream_ = static_cast<CUstream_st*>(stream);
13    }
14    // ...
15    // ...
```

```

16     if (memcpy_kind == MemcpyKind::kMemcpyCPU2CUDA) {
17         if (!stream_) {
18             cudaMemcpy(dest_ptr, src_ptr, byte_size,
cudaMemcpyHostToDevice);
19         } else {
20             auto e = cudaGetLastError();
21             cudaMemcpyAsync(dest_ptr, src_ptr, byte_size,
cudaMemcpyHostToDevice, stream_);
22         }
23     }
24 }

```

在 `cu_alloc` 的 `memcpy` 方法中我们再调用 `cudaMemCpy` 方法完成数据从 CPU 到 CUDA 的拷贝，换句话说我们将原来 `buffer_` 中的数据通过 `cudaMemcpy` 拷贝到了新创建的显存块 `cudaBuffer` 的 `ptr` 中。

```

1  this->buffer_ = cu_buffer;

```

当数据拷贝同步完成之后，我们再将该张量的类内变量 `buffer_` 指向 `cu_buffer`，从此完成了张量数据到显存的过程。

单元测试

```

1  TEST(test_tensor, to_cu) {
2      using namespace base;
3      auto alloc_cpu = CPUDeviceAllocatorFactory::get_instance();
4      tensor::Tensor t1_cpu(DataType::kDataTypeFp32, 32, 32, true,
alloc_cpu);
5      ASSERT_EQ(t1_cpu.is_empty(), false);
6      float* p1 = t1_cpu.ptr<float>();
7      for (int i = 0; i < 32 * 32; ++i) {
8          *(p1 + i) = 1.f;
9      }
10
11     t1_cpu.to_cuda();
12     float* p2 = new float[32 * 32];
13     cudaMemcpy(p2, t1_cpu.ptr<float>(), sizeof(float) * 32 * 32,
cudaMemcpyDeviceToHost);
14     for (int i = 0; i < 32 * 32; ++i) {
15         ASSERT_EQ(*(p2 + i), 1.f);
16     }
17     delete[] p2;
18 }

```

我们先创建了一个 CPU 类型的张量 `t1_cpu`，并将其中的所有值置为1。随后我们调用 `to_cu()` 方法将数据同步到显存上，最后再用单元测试逐个元素检查其中的值是否均为1。

